

AWS - CC_POPEYES

Sistema de Gestión de Pedidos

Arquitectura cloud — backend **serverless multi-cloud**
(AWS + GCP) orientado a eventos.

SERVERLESS

EVENT-DRIVEN

MULTI-TENANT

MULTI-CLOUD

INTEGRANTES

Silva Reyes, Santiago

Cama Ardiles, Santiago Aldebarán

Ayala Pereda, Aracely Yobely

Ladera La Torre, Fabricio Godofredo

Ricapa Montoya, Steve Bryan

¿Qué es AWS-CC_POPEYES?

Backend serverless para un **Sistema de Gestión de Pedidos de Popeyes**, desplegado completamente en AWS. Los clientes realizan pedidos desde el portal web o desde Rappi, y el personal de tienda los procesa paso a paso hasta su entrega final.

11 funciones Lambda

6 dominios funcionales

6 tablas DynamoDB

01

Serverless

100% AWS Lambda + API Gateway, sin servidores que administrar.

02

Orientado a eventos

EventBridge enruta eventos de dominio entre servicios.

03

Multi-tenant

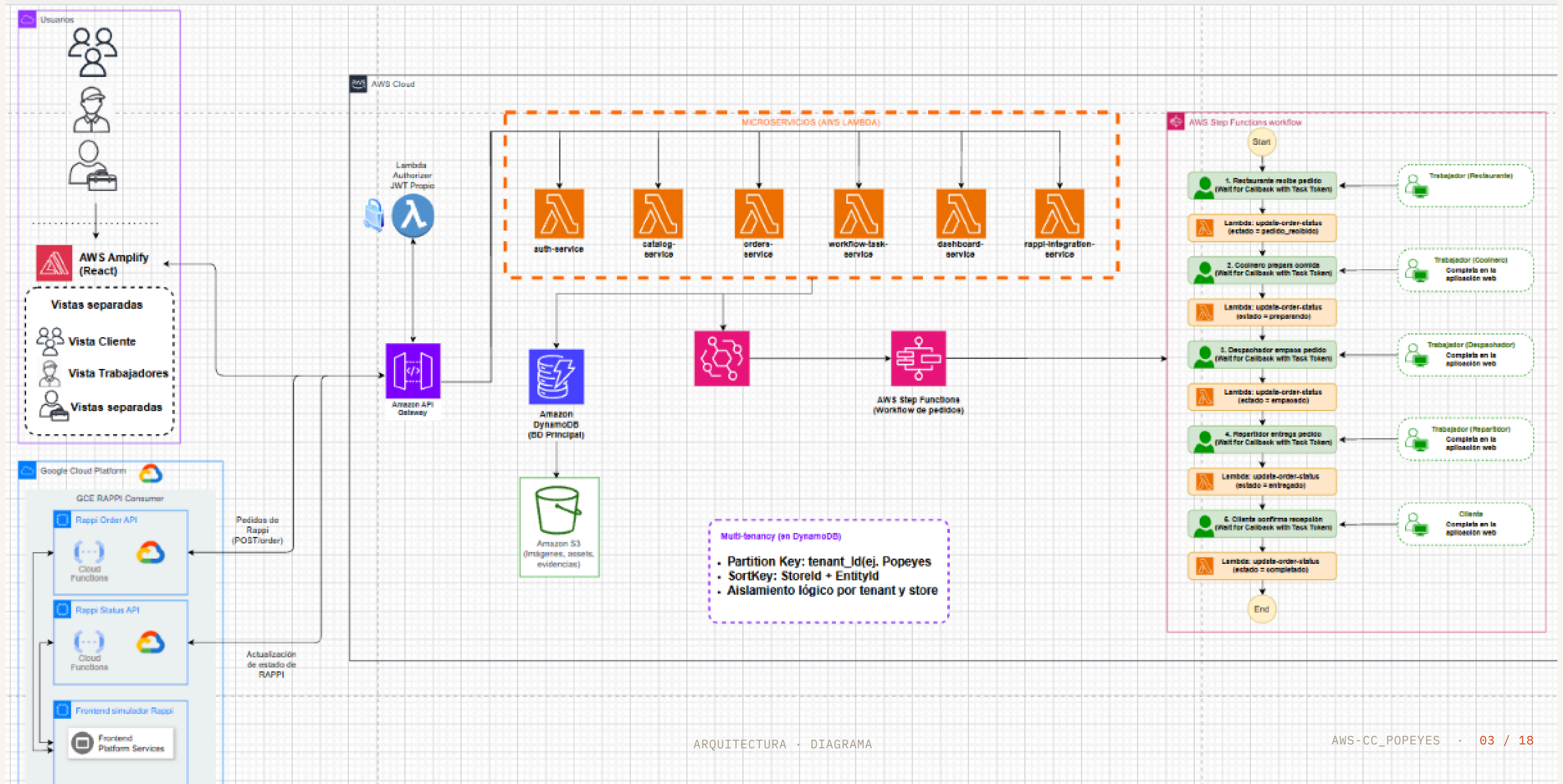
tenantId presente en todas las tablas principales.

04

Multi-cloud

AWS como backend + GCP para la integración con Rappi.

Arquitectura general del sistema



¿Por qué multinube?

El sistema usa **AWS** como backend principal y **GCP** como integración externa, simulando la plataforma real de Rappi — un caso típico de interoperabilidad entre nubes en ecosistemas de delivery.

AWS — Backend principal

- › Lógica de negocio completa (Lambda + FastAPI)
- › Persistencia de datos (DynamoDB, 6 tablas)
- › Orquestación del workflow (Step Functions)
- › Bus de eventos de dominio (EventBridge)
- › Autenticación propia vía JWT (sin Cognito)

GCP — Integración externa (Rappi)

- › Recibe pedidos vía **POST /orders/rappi** (x-api-key)
- › No requiere JWT: el sistema externo no tiene usuarios
- › **notifyRappiStatus** escucha OrderStatusChanged
- › Notifica cambios de estado al webhook de Rappi
- › Solo actúa cuando **origin = RAPPI**

Herramientas y servicios utilizados

RUNTIME Y FRAMEWORK

Runtime

Python 3.11

Framework HTTP

FastAPI + Mangum

IaC

Serverless Framework v3

Packaging

serverless-python-requirements

SERVICIOS AWS

- API Gateway HTTP API
- AWS Lambda (11 funciones)
- Lambda Authorizer (JWT)
- DynamoDB (6 tablas, on-demand)
- EventBridge
- Step Functions

CloudWatch · IAM (roles de ejecución) · S3 (assets)

VARIABLES DE ENTORNO CLAVE

STAGE

REGION

JWT_SECRET

RAPPI_API_KEY

EVENT_BUS_NAME

ORDER_WORKFLOW_ARN

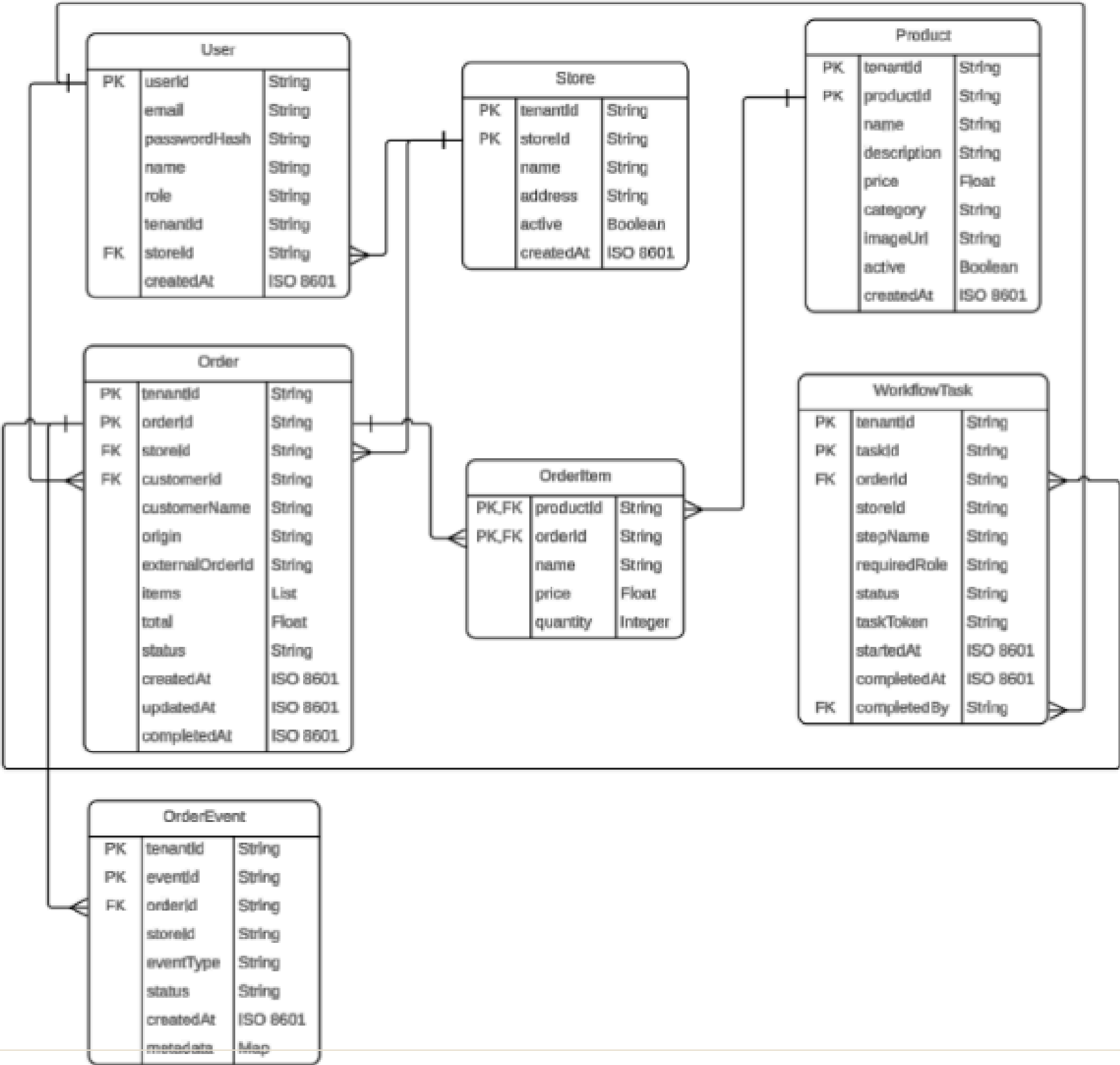
7 entidades del sistema

6 tablas DynamoDB + **OrderItem** como entidad derivada de los ítems embebidos en Order.

ENTIDAD	TABLA DYNAMODB	PARTITION KEY
User	popeyes-users	userId
Store	popeyes-stores	tenantId + storeId
Product	popeyes-products	tenantId + productId
Order	popeyes-orders	tenantId + orderId
OrderItem derivada	<i>(embebido en Order)</i>	orderId + productId
WorkflowTask	popeyes-workflow-tasks	tenantId + taskId
OrderEvent	popeyes-order-events	tenantId + eventId

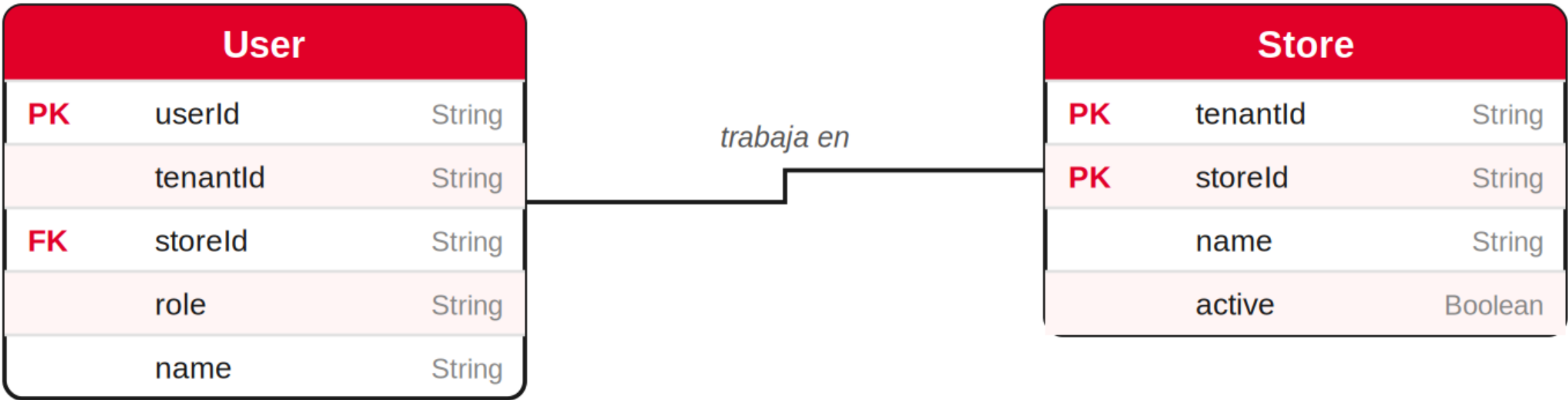
Diagrama entidad-relación completo

VISTA GENERAL



User → Store

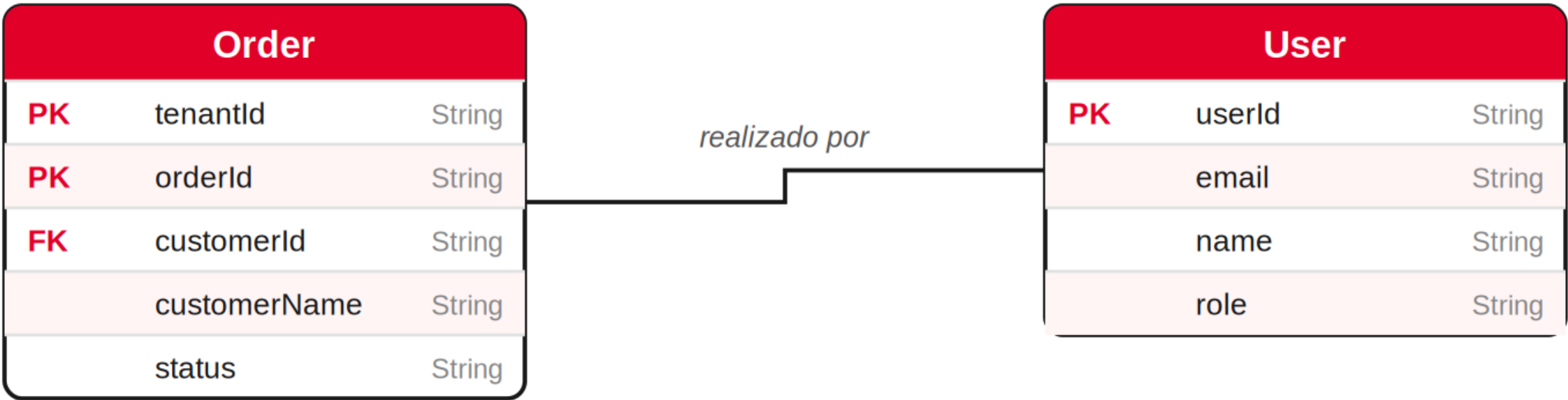
Cada usuario de tienda pertenece a una sede identificada por **storeId**. Define a qué local está asignado cada miembro del personal dentro del tenant.



N : 1 pertenece a

Order → User

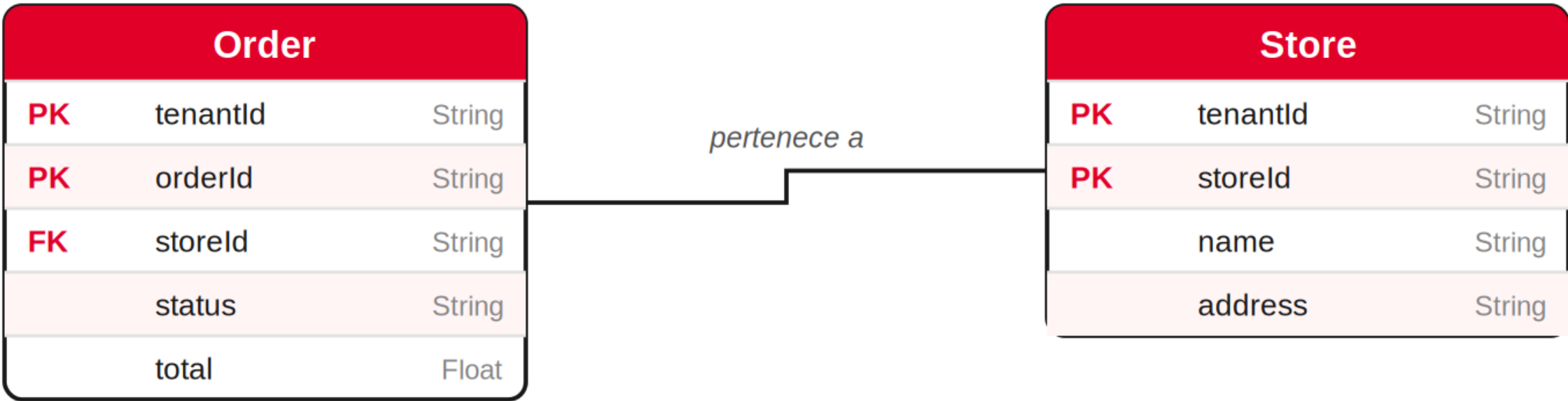
Cada orden es realizada por un cliente: el campo **customerId** referencia el **userId** del usuario que generó el pedido.



N : 1 realizado por

Order → Store

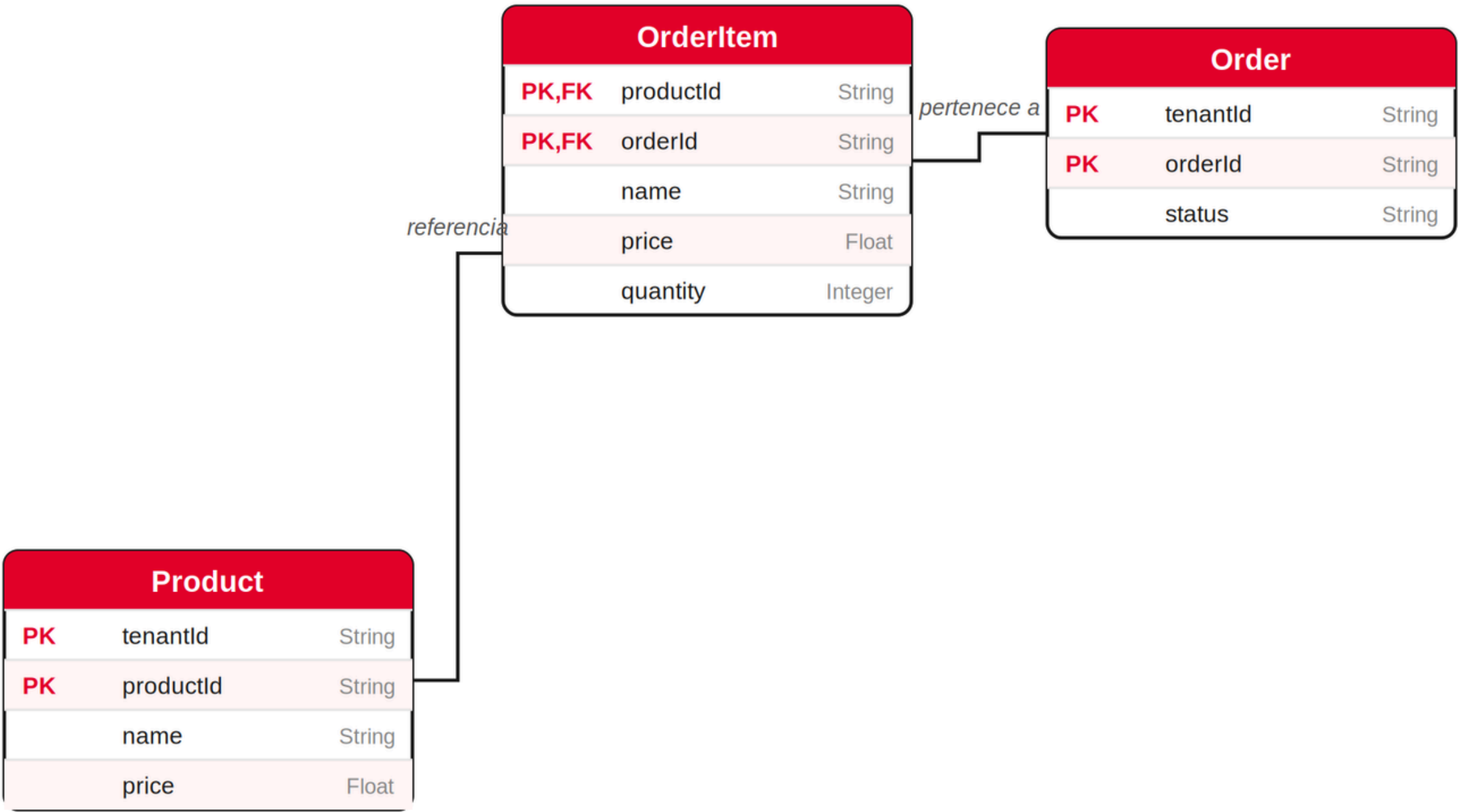
Cada orden **pertenece a una tienda** concreta vía **storeId**, donde el pedido se prepara, empaqueta y despacha.



N : 1 despachada en

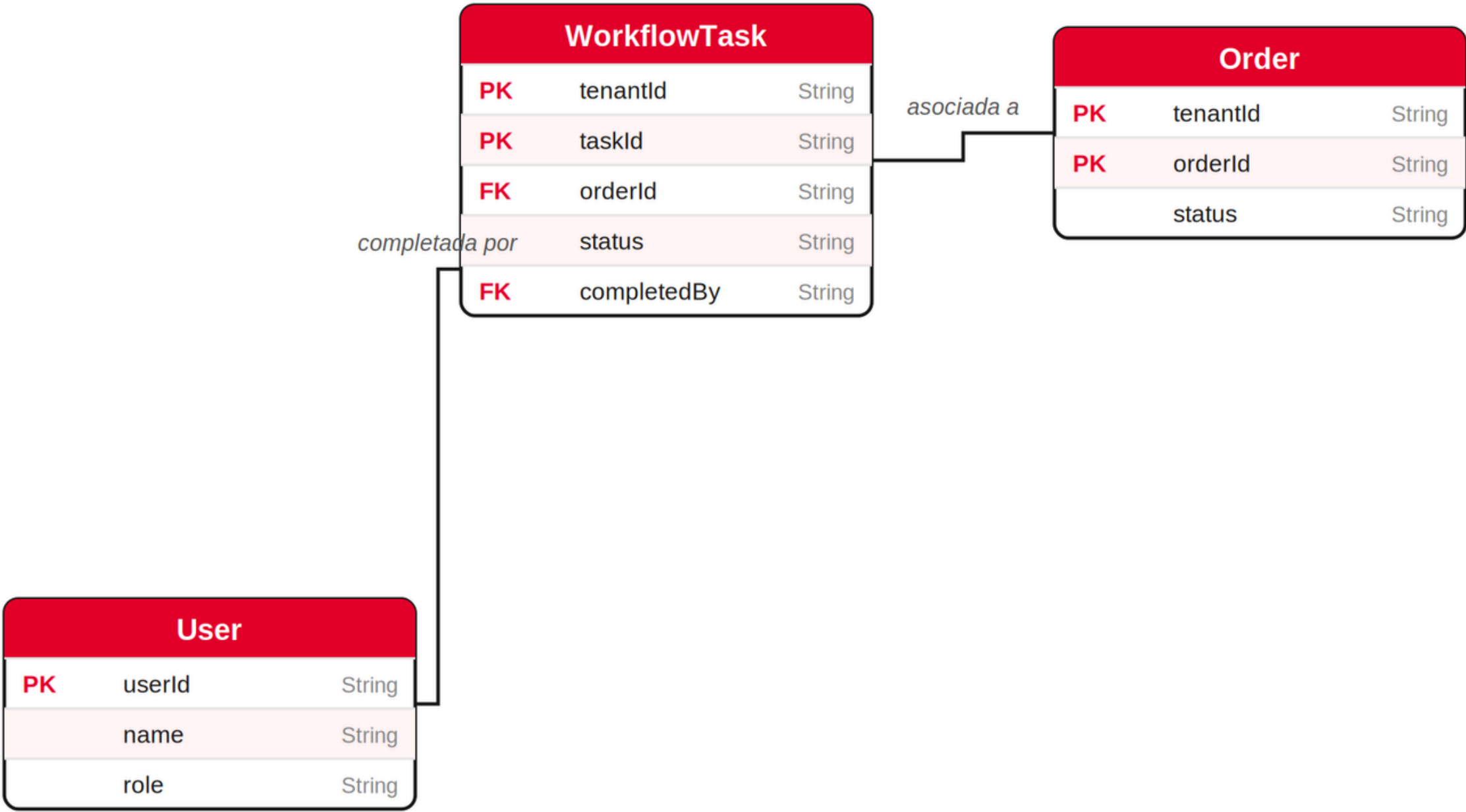
OrderItem → Order y Product

Cada ítem de pedido referencia su **orderId** y el **productId** pedido. Entidad derivada de los ítems embebidos en Order.



WorkflowTask → Order y User

Cada tarea del workflow está asociada a una orden (`orderId`) y es completada por un usuario (`completedBy`).



OrderEvent → Order

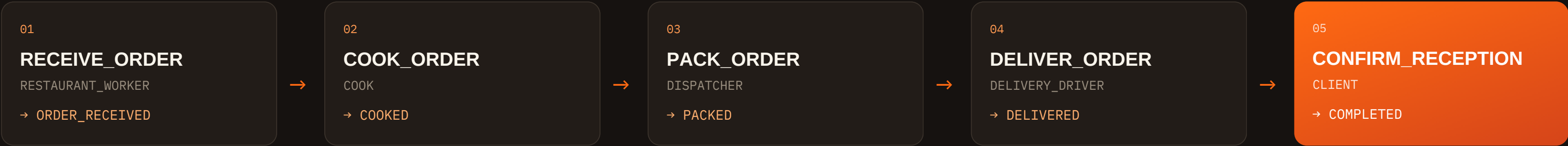
Cada evento de auditoría referencia su **orderId**, trazando el historial completo de cambios de estado del pedido.



N : 1 audita

Orquestación con Step Functions

5 etapas humanas con patrón Wait for Callback (Task Token).



CICLO DE CADA ETAPA

`createHumanTask` crea la tarea → el trabajador la completa vía `POST /tasks/{id}/complete` → `SendTaskSuccess` avanza la State Machine → `updateOrderStatus` publica `OrderStatusChanged`.

Inicio: `ORDER_CREATED`
Cierre: `closeOrder` registra `completedAt` al finalizar las 5 etapas.

Roles de usuario y ciclo de vida

ROLES DEL SISTEMA

CLIENT	Realiza pedidos y confirma recepción
RESTAURANT_WORKER	Recibe y valida el pedido
COOK	Prepara el pedido
DISPATCHER	Empaqueta para envío
DELIVERY_DRIVER	Entrega el pedido
ADMIN	Acceso total: dashboard, seed, etc.

ESTADOS DEL PEDIDO

1	ORDER_CREATED
2	ORDER_RECEIVED
3	COOKED
4	PACKED
5	DELIVERED
6	COMPLETED

Aislamiento de datos y autenticación

Multi-tenancy

- › **tenantId** identifica la franquicia o marca (demo: popeyes)
- › **storeId** identifica la sede específica (demo: store-001)
- › Toda tabla principal incluye **tenantId** en su partition key o atributo
- › Las rutas autenticadas operan exclusivamente sobre el **tenantId / storeId** extraídos del JWT

Autenticación y autorización

- › Lambda Authorizer tipo **request**, valida Bearer JWT (sin Cognito)
- › JWT firmado con **HS256** vía PyJWT; claims: userId, role, tenantId, storeId
- › Cada endpoint verifica el **role** del JWT contra los roles permitidos
- › Las tareas del workflow validan **role** vs **requiredRole**
- › Bootstrap: el primer registro puede ser ADMIN; luego se fuerza CLIENT

Del pedido a la entrega: flujo end-to-end

- 1 Cliente/Rappi crea el pedido (**POST /orders** o **/orders/rappi**)
- 2 Se valida JWT (web) o **x-api-key** (Rappi)
- 3 Se persiste la Order (**status=ORDER_CREATED**) y se registra OrderEvent
- 4 Se publica **OrderCreated** en EventBridge → arranca Step Functions
- 5 Por cada etapa: se crea WorkflowTask, el rol la completa, y **SendTaskSuccess** avanza el workflow
- 6 Si **origin=RAPPI**, notifyRappiStatus notifica a GCP en cada cambio de estado
- 7 **closeOrder** registra **completedAt** al finalizar todas las etapas

Lo que demuestra el proyecto

- 01** Arquitectura 100% serverless, escalable y sin gestión de infraestructura.
- 02** El patrón EDA + Wait for Callback modela workflows humanos de forma robusta.
- 03** Diseño multi-tenant desde la base de datos, listo para múltiples franquicias.
- 04** La integración multi-cloud (AWS + GCP) demuestra interoperabilidad real con Rappi.